

Лаба по сортам

Цель данной лабораторной работы – посмотреть на алгоритмы с различной асимптотикой, научиться анализировать время работы алгоритмов и включать разные степени оптимизации.

Заодно это практика написания кода по описанному алгоритму. Рекомендую писать код самостоятельно. Если напишете быстро, то много времени не потратите, а если закопаетесь – ну, значит, практика вам явно пойдет на пользу.

Для сортировки советую использовать написанные ранее контейнеры `subvector` (добавив к ним функции `sort/sort_bubble/sort_heap/...`) и рассматривать массивы до 4-8 Мб. Если расчетное время одного графика будет больше часа – окей, можно взять поменьше.

Как и в предыдущих лабах, постарайтесь сделать свою жизнь легче заранее. Планируйте код таким образом, чтобы с ним было удобно работать – меньше переписывать между пунктами или при замене размера контейнера. Используйте свои наработки и готовые куски кода. По возможности, автоматизируйте процесс перестроения графиков – запись в файл, построение графиков скриптом на питоне. Можете заранее прочитать весь текст лабы, чтобы спланировать работу над кодом, вы таки не поверите, как это помогает.

Чтобы сдать эту лабу, вам нужно прислать мне **скриншоты графиков – по одному на каждый пункт**.

0. Пузырек и его товарищи

Начнем с простого. Напишите три разных сортировки с временной сложностью $O(N^2)$. Это может быть пузырек, вставка, шейкер, простой выбор, непростой выбор или что угодно, что вы еще науглите/найдете в лекциях.

Постройте график зависимости времени работы от размера массива с тремя кривыми – по одной для каждой сортировки.

Докажите, что сложность этих алгоритмов действительно $O(N^2)$.

Далее подсказки в режиме спойлера, светло-серым по белому (для новичков в компах и интерфейсах – выделяйте текст мышкой, и он или сразу станет читаемым, или копируйте в любой текстовый редактор).

Доказать сложность в данном случае проще всего в двойном логарифмическом масштабе. Если $t = O(N^2)$, то тогда $t \sim C * N^2 \Rightarrow \ln t = \ln C + 2 * \ln N$. То есть параболы в координатах t и N превратятся в прямые в координатах $\ln t$ и $\ln N$. Наклон прямых будет говорить о максимальной степени N , а высота над началом координат – о коэффициенте при этой максимальной степени.

1. Пузырек, но быстрее

А теперь лезем в настройки компилятора.

Если вы компилируетесь и запускаетесь из консоли, то добавляйте при компиляции флаги -O0, -O1, -O2 и -O3 соответственно. Если из среды разработки, то вам нужно найти галочки с такими же названиями в настройках компилятора.

Постройте график с четырьмя кривыми для одной из сортировок за $O(N^2)$ – по одной кривой на каждую степень оптимизации.

Если вы ничего такого не делали, а у вас пошла стопка нулей, см ниже `vv`

Иногда оптимизацию O3 надо обманывать, чтобы получить графики такого типа. Если этот флаг включен, компилятор начинает всеми правдами и неправдами ускорять ваш код. В некоторых редких случаях он может его даже сломать (очень редко, очень сложно такое поймать – обычно это или при ассемблерных вставках, или при адском жонглировании `r-value`). Если что, O2 обычно оптимизирует примерно так же, но гарантированно не ломает ничего.

В частности, O3 вырезает куски кода, если они вычисляют значения, которые потом не выходят за пределы программы. Ну, например, всю вашу сортировку – логично, если на экран вы выводите только время. Обходить это можно разными способами – можно выводить что-то из массива на экран, можно отмечать ваш контейнер как `volatile` (ключевое слово перед объявлением переменной). Поэкспериментируйте. Игры в армрестлинг с O3 помогают неплохо понять некоторые ее аспекты.

```
binary(a_lab_rat.u);
```

2. А теперь настоящие быстрые сортировки

Напишите три разных сортировки с временной сложностью $O(N \log N)$. Это может быть слияние, куча, Хоар или что угодно, что вы еще науглите/найдете в лекциях.

Постройте график зависимости времени работы от размера массива с тремя кривыми – по одной для каждой сортировки.

Докажите, что сложность этих алгоритмов действительно $O(N \log N)$.

Для того, чтобы это доказать, вам нужно свести график к какому-нибудь линейному виду. Предлагаю построить зависимость $t/N \log N$ от N . Как вы догадываетесь, должна получиться константа. Если придумаете что-то более элегантное, как двойной логарифмический масштаб для полинома – пишите и рассказывайте.

3. $O(N^2)$ vs $O(N \log N)$

Весьма напрашивающийся пункт. Нарисуйте кривые из пунктов 0 и 2 на одном графике. Возможно, вам для этого придется какие-то из них пересчитать, чтобы масштабы соответствовали. И не запутайтесь в том, какую оптимизацию используете – она может быть любой, но должна быть одинаковой для всех расчетов в этом пункте.

4. Зависимость от начальных данных

Сравните время работы алгоритмов на отсортированном массиве, рандомном массиве и массиве, отсортированном в обратную сторону. То есть лучший, средний и худший случай.

Для каждого из шести реализованных алгоритмов постройте по три кривых – лучший, средний и худший случай. Подумайте, как их сгруппировать, чтобы наиболее наглядно продемонстрировать разницу – в этом пункте может получиться несколько картинок.

Опять же – не запутайтесь во флагах и галочках, оптимизация должна быть одинаковой.

5. Небольшие массивы

В этом пункте вам нужно посмотреть на такую неочевидную штуку, как время работы шести предыдущих сортировок на небольших массивах (примерно 1 – 1000 элементов). Суть в том, что асимптотика начинает работать при очень больших N – неспроста во всех выкладках и оценках мы так за просто отбрасывали все слагаемые, кроме тех, что указаны в самой асимптотике. А при малых N эти слагаемые и коэффициенты при них вылезают, что может сильно повлиять на итоговое время работы.

На этом графике тоже должно быть шесть кривых, по одной для каждого из реализованных алгоритмов. Про оптимизацию не забывайте.

Если у вас получается стопка нулей, облако точек или ступенчатое чудовище, см ниже 

Время работы сортировки на небольших массивах (особенно до 10 элементов) может быть меньше чувствительности таймера. Не верьте `chrono`, что его `nanoseconds` это честные наносекунды – он вполне может считать в десятках тысяч или миллионах наносекунд. Таймер штука системная, и на разных машинах/ОС его точность может отличаться.

В этом случае повторяйте сортировку и усредняйте по количеству повторений – 1000, 10000, 1000000 – сколько понадобится, чтобы графики стали гладкими. Да, это одна из причин автоматизировать процесс отрисовки графиков. Я предупреждала.

Не забывайте, что время работы алгоритма зависит от входных данных – если вы раз за разом сортируете уже отсортированный массив, то это будет не совсем то время, которое нам нужно. Или совсем не то, тут уж какой алгоритм. Если вы повторяете сортировку – подумайте, как каждый раз сортировать новый произвольный массив.

Особенно учитывая, что `random` – штука медленная. Если вы вызываете его в рамках таймера, он перекроет все другие эффекты, которые вы пытались посмотреть. Выносите его за пределы таймера и не стесняйтесь жрать дополнительную память, в этом пункте у нас мелкие массивы.

6*. Зависимость от типа данных

Если вы любитель пунктов со звездочкой, то вот вам один. Скорее всего, в предыдущих пунктах вы сортировали инты. А попробуйте отсортировать массивы других типов данных и сравнить время работы.

Будет наиболее наглядно, если на одном графике изобразить кривые для времени сортировки разных типов – char, short int, int, long int, long long int, а также нескольких структур. Особенно интересны «тяжелые» структуры – с большим количеством полей и статическими массивами.

А еще оптимизация, ага.

7*. Сортировка списков

Если вы добрались до второго пункта со звездочкой и желаете от жизни странного.

Отсортируйте односвязный список (контейнер `subforwardlist`) методом пузырька. Причем меняя не поля `data` (неспортивно и неинтересно с точки зрения практического применения), а именно перекладывая туда-сюда указатели. Измерьте время работы, постройте график, сравните со временем сортировки массива такого же размера – асимптотику и константу.

Попробуйте подобрать алгоритм сортировки, который отсортирует ваш список за $O(N \log N)$. Теперь вы уже умеете проверять асимптотику, вас не обманешь.

Оптимизация.